

Search Engine for Scientific Publications

Technical Report

Master's in Biomedical Engineering
Medical Informatics

Information Retrieval and Processing
2025/2026

Group 1:

Daniela Coelho (pg59872)
Daniela Rodrigues (pg59757)
Sara Chiapperino (e12997)

Table of Contents

1	Introduction	1
2	Fundamental Theoretical Concepts	2
2.1	Text Preprocessing	2
2.2	Inverted Index	2
2.3	Boolean Model	2
2.4	Vector Space Model and TF-IDF	3
2.5	Cosine Similarity	3
2.6	Term-Document Matrix	3
3	System Architecture	4
3.1	Technologies Used	6
4	Development	7
4.1	Data Collection Module — Scraper	7
4.2	Document Loader	7
4.3	Information Retrieval Engine	8
4.3.1	Text Preprocessing Pipeline (<code>text_pipeline.py</code>)	8
4.3.2	Inverted Index (<code>inverted_index.py</code>)	8
4.3.3	Term-Document Matrix (<code>term_document_matrix.py</code>)	9
4.3.4	TF-IDF Engine (<code>tfidf.py</code>)	9
4.3.5	SearchEngine Orchestrator (<code>search_engine.py</code>)	10
4.4	REST API (<code>fastapi_app.py</code>)	10
4.5	Web Interface (<code>frontend/index.html</code>)	11
4.6	Storage Module (<code>storage/</code>)	13
4.7	Testing	13
5	Limitations	14
6	Conclusion	14

Abstract

This report describes the development of a search engine for scientific publications applied to RepositóriUM, the institutional repository of the University of Minho.

The project was developed within the scope of the curricular unit *Information Retrieval and Processing* (PRI), during the academic year 2025/2026, with the main objective of designing and implementing, from scratch, a system capable of collecting, preprocessing, indexing, and efficiently retrieving scientific information.

The system integrates all the fundamental components of a modern Information Retrieval engine: a Selenium-based web scraper for automated metadata collection, a configurable NLP text preprocessing pipeline supporting both stemming and lemmatisation, an inverted index with skip-pointer-optimised postings lists, a term-document matrix supporting binary, TF, and TF-IDF modes for Boolean retrieval, and a TF-IDF ranking engine with two interchangeable backends — a custom implementation and a `scikit-learn` implementation. A REST API built with FastAPI exposes all search functionality, and a single-page web interface provides an interactive front end, including an educational panel that demonstrates the inner workings of the IR algorithms in real time.

1 Introduction

Information Retrieval (IR) is the discipline concerned with identifying, within a document collection (corpus), those items that best satisfy a user's information need. Unlike database retrieval, which seeks exact matches to structured queries, IR systems operate on natural language and must deal with ambiguity, synonymy, and variation in expression.

Modern IR systems address these challenges through a pipeline of techniques: automated data collection, linguistic normalisation of text, construction of efficient index structures, and application of relevance models that rank results according to their estimated utility for the user.

The present project applies this pipeline to RepositóriUM, the institutional repository of the University of Minho, which hosts thousands of theses, dissertations, scientific articles, and other academic documents. The goal is to build a fully functional search engine that allows any user to query this collection efficiently and receive results ranked by relevance.

The system was designed to fulfil the following objectives:

- Implement a web scraping module for automated metadata collection from RepositóriUM;
- Build an inverted index and perform Boolean searches using postings lists;
- Apply text preprocessing techniques (tokenisation, stemming, lemmatisation, stop-word removal);
- Compute TF-IDF weights and rank results using cosine similarity;
- Develop a REST API using FastAPI and a web interface for system exposure;
- Integrate full-text content from PDF documents to enrich the index.

During the development of this project, artificial intelligence tools were used to support code implementation, report writing, and the preparation of presentation materials. Their role was strictly auxiliary, with all decisions, implementation choices, and the design of visualizations remaining entirely the responsibility of the authors.

The full source code of the project is available at: <https://github.com/pg59757/IR-class-project>

2 Fundamental Theoretical Concepts

This section presents the main theoretical concepts that support the development of the Information Retrieval engine implemented in this project. These foundations help explain the implementation decisions described in the following sections, as well as the functioning of the search models used.

2.1 Text Preprocessing

Text preprocessing is the first stage of any Information Retrieval system, responsible for transforming raw text into a normalized representation. The main operations applied were:

- **Tokenization:** division of text into linguistic units (tokens), typically words.
- **Stopword removal:** elimination of highly frequent and semantically weak terms.
- **Stemming:** reduction of words to their morphological roots, using the Porter Stemmer for English and the Snowball Stemmer for Portuguese.
- **Lemmatization:** transformation of words into their canonical form (lemma), using the WordNet Lemmatizer with POS annotation.

These operations reduce vocabulary dimensionality and increase the robustness of the indexing process.

2.2 Inverted Index

The inverted index is the central data structure of IR systems. For each term in the vocabulary, a *postings list* is stored containing:

- **df** (document frequency): number of documents in which the term occurs;
- **postings:** pairs (`doc_id`, `tf`) with the term frequency in the document;
- **skip pointers:** pointers that accelerate list intersection.

This structure enables answering queries without scanning the entire collection, ensuring efficiency in Boolean search and vector model construction.

2.3 Boolean Model

The Boolean model allows combining terms using logical operators AND, OR, and NOT. The query is interpreted by a syntactic parser based on a formal grammar, and the result corresponds to the set of documents that exactly satisfy the Boolean expression.

The AND operator is implemented through the intersection of postings lists, OR through their union, and NOT through document exclusion.

2.4 Vector Space Model and TF-IDF

In the vector space model, each document is represented as a vector in a high-dimensional space, where each dimension corresponds to a vocabulary term.

The weight of each term is computed using the TF-IDF metric:

$$\text{tf}_{\log} = 1 + \log(\text{tf})$$

$$\text{idf} = \log\left(\frac{N + 1}{df + 1}\right) + 1$$

where N is the total number of indexed documents and df the document frequency of the term.

2.5 Cosine Similarity

The relevance of a document with respect to a query is computed using cosine similarity:

$$\cos(\theta) = \frac{\vec{d} \cdot \vec{q}}{\|\vec{d}\| \|\vec{q}\|}$$

This metric evaluates the angular proximity between vectors, being independent of their magnitude.

2.6 Term-Document Matrix

The term-document matrix represents the collection in matrix form, where each row corresponds to a term and each column to a document. Depending on the selected mode, the matrix may store binary values, TF, or TF-IDF.

Boolean operations can be implemented as vector operations: AND as element-wise multiplication, OR as addition, and NOT as subtraction, enabling a matrix-based interpretation of the classical Boolean model.

3 System Architecture

The system follows a layered architecture composed of five distinct layers, illustrated in Figure 1, each with a well-defined responsibility:

1. **Data Collection** (`scraper/`): automated metadata acquisition from RepositóriUM via Selenium.
2. **Information Retrieval Engine** (`search/`): the core backend, comprising the preprocessing pipeline, inverted index, term-document matrix and TF-IDF engine.
3. **Document Loader** (`document_loader.py`): merges scraper metadata with full-text content extracted from PDFs.
4. **REST API** (`api/`): FastAPI application that exposes all search functionality as HTTP endpoints.
5. **Web Interface** (`frontend/`): single-page application providing an interactive search interface with an educational panel.

Frontend (HTML / CSS / JavaScript)
REST API (FastAPI + Pydantic)
IR Engine (Preprocessor, Inverted Index, TF-IDF, Boolean Search, Term-Document Matrix)
Document Loader (<code>scraper_results.json</code> + <code>jsons/*.json</code>)
Storage (SQLite + JSON)
Data Collection (Selenium)

Figura 1: Layered architecture of the information retrieval system.

The data flow begins in the scraper, which exports metadata to `scraper_results.json`. The document loader then enriches each document with full-text content from the corresponding PDF JSON files (when available), producing a unified list of document dictionaries. This list is passed to the `SearchEngine`, which constructs all internal indexes. The FastAPI application instantiates two `SearchEngine` instances — one with the custom TF-IDF backend and one with the `scikit-learn` backend — and exposes them through HTTP endpoints consumed by the frontend.

The project is organised as follows:

```

PRI_G1_Project_v2/
src/
    config.py                # Centralised configuration (env-overridable)
    document_loader.py       # Merges scraper + PDF JSON data
    scraper/
        scraper.py           # Selenium-based DSpace 8 scraper
        main.py              # Scraper entry point
        scraper_results.json
    search/
        text_pipeline.py     # NLP preprocessing pipeline
        inverted_index.py    # Inverted index with skip pointers
        boolean_search.py    # Boolean engine (AND/OR/NOT)
        term_document_matrix.py # Term-document matrix (binary/tf/tfidf)
        tfidf.py             # TF-IDF custom + sklearn + cosine similarity
        search_engine.py     # Unified orchestrator
        classifier.py        # Naive Bayes topic classifier
        query_utils.py       # Query parsing utilities
        evaluation.py        # Evaluation metrics
        run_engine.py        # CLI entry point for the IR engine
    api/
        fastapi_app.py       # REST API
    frontend/
        index.html           # Single-page web interface
    storage/
        database.py          # SQLite persistence layer
        populate_db.py       # Database population script
docs/
    PRI_G1_Report_v2.pdf     # Technical report
jsons/
    # Full-text PDF JSONs (pdf01.json ... pdf20.json)
pdfs/
    pdfs/                   # Original PDF files (pdf01.pdf ... pdf20.pdf)
    pdftotext/              # Plain-text extractions (pdf01.txt ... pdf20.txt)
tests/
    preprocessor/
        test_preprocessor.py
    test_api.py
    test_tfidf.py
    test_scraper.py
    test_index_search.py
    test_classifier.py
    test_database.py
    test_query_utils.py
data/
    repositiorium.db         # SQLite database
conf/test.py
requirements.txt

```

3.1 Technologies Used

Tabela 1: Technologies used per system layer.

Layer	Technologies
Data Collection	Python 3.11, Selenium, ChromeDriver
NLP / IR Engine	NLTK (tokenisation, Porter/Snowball stemming, WordNet lemmatisation), NumPy, scikit-learn
REST API	FastAPI, Uvicorn, Pydantic
Web Interface	HTML5, CSS3, JavaScript (vanilla)
Storage	JSON, SQLite
Configuration	Environment variables with <code>IR_</code> prefix
Testing	pytest, pytest-cov

4 Development

4.1 Data Collection Module — Scraper

The scraper (`scraper.py`) is responsible for the automated collection of scientific metadata from RepositóriUM, the University of Minho’s DSpace 8 repository. It was implemented using the **Selenium** framework to navigate the Angular-based web interface, which relies on dynamic JavaScript rendering and is not amenable to simple HTTP-based scraping.

During initialisation, the module auto-detects the Chrome executable across Windows, Linux, and macOS, ensuring portability across heterogeneous environments. Once configured, Selenium connects to RepositóriUM and navigates through the collection pages, collecting the URLs of individual publications.

For each publication, the scraper visits the full metadata view (`/full`) and extracts structured information from the DSpace metadata table by mapping Dublin Core field identifiers to a normalised internal JSON structure:

Tabela 2: Metadata fields extracted by the scraper.

Dublin Core Field	JSON Key	Description
<code>dc.title</code>	<code>title</code>	Publication title
<code>dc.date.issued</code>	<code>year</code>	Year of publication
<code>dc.identifier.doi</code>	<code>doi</code>	DOI identifier
<code>dc.contributor.author</code>	<code>authors</code>	List of authors
<code>dc.description.abstract</code>	<code>abstract</code>	Abstract text
—	<code>document_link</code>	URL to the PDF

The scraper supports paginated navigation with a configurable `MAX_ITEMS` parameter and includes error-handling mechanisms for network failures, asynchronous loading timeouts, and temporarily unavailable page elements. The collected metadata is exported as `scraper_results.json` for use by the IR engine.

4.2 Document Loader

The document loader (`document_loader.py`) acts as a data fusion layer between two complementary sources:

1. `scraper_results.json`: metadata for all scraped documents (title, authors, year, DOI, abstract, PDF link).
2. `jsons/pdfNN.json`: full-text content extracted from the PDFs of the first 20 documents, stored as sentence-segmented JSON files.

The PDF JSON files were produced through a manual process complementary to the automated scraper. For the first 20 documents in the collection, the corresponding PDFs were downloaded directly from RepositóriUM and converted to plain text using a PDF-to-text extraction tool. The resulting text files were then segmented into sentences and serialised as JSON (one sentence per list entry), yielding the files `jsons/pdf01.json` through `jsons/pdf20.json`.

This manual step was necessary because RepositóriUM does not expose full-text content through its metadata API, and automated bulk PDF download raises reliability and rate-limiting concerns.

The loader iterates over the scraper metadata and, for each document with a corresponding PDF JSON file, reconstructs the full text from the sentence list and appends it to the document as a `full_text` field. Documents beyond the first 20 retain an empty `full_text` and are indexed solely on their title and abstract. The merged documents are passed directly to `SearchEngine.build()`.

The `full_text` field is included among the indexed fields (`["title", "abstract", "full_text"]`), enriching the index and the TF-IDF vectors for documents that have associated PDF content.

4.3 Information Retrieval Engine

The IR engine is implemented as a set of loosely coupled modules, orchestrated by the `SearchEngine` class (`search_engine.py`), which provides a unified interface for building indexes and executing searches.

4.3.1 Text Preprocessing Pipeline (`text_pipeline.py`)

The preprocessing pipeline is implemented in the `Preprocessor` class, configured via a `PreprocessorConfig` dataclass. The pipeline applies the following operations in order:

1. Unicode normalisation and accent/punctuation removal;
2. Word tokenisation (`nltk.word_tokenize`);
3. Stop-word filtering (configurable; English and Portuguese lists);
4. Minimum token length filter (`MIN_TOKEN_LENGTH`);
5. Stemming — Porter Stemmer for English, Snowball Stemmer for Portuguese;
6. Lemmatisation — WordNet Lemmatizer with POS tagging for increased morphological accuracy.

Three factory functions simplify instantiation: `make_stemming_preprocessor()`, `make_lemmatisation_preprocessor()` and `make_bare_preprocessor()`. A fourth mode (`full`) applies both lemmatisation and stemming sequentially. The preprocessor operates both on individual queries (`process()`) and on multi-field document dictionaries (`process_document()`), concatenating the content of the configured fields before tokenisation.

4.3.2 Inverted Index (`inverted_index.py`)

The inverted index maps each vocabulary term to a `PostingsList` object, which stores:

- **df**: document frequency (number of documents containing the term);
- **postings**: a sorted list of `Posting` objects, each holding `doc_id`, term frequency `tf`, and token position list;
- **skips**: a dictionary of skip pointers, keyed by position index in the postings list.

Skip pointers are inserted at intervals of $\lfloor \sqrt{n} \rfloor$ where n is the postings list length, and are only added for lists with at least 3 entries. During AND intersection, the algorithm checks whether a skip is available and uses it if the target entry does not exceed the current entry of the other list, reducing the number of comparisons.

The index also supports author-based retrieval via `search_by_author()`, which performs a case-insensitive substring match over the authors field of all indexed documents. Incremental document addition is supported through `add_document()`, which extends the index without requiring a full rebuild.

4.3.3 Term-Document Matrix (`term_document_matrix.py`)

The term-document matrix is implemented using a dense NumPy array of shape $(|V|, N)$, where rows correspond to vocabulary terms and columns to documents. Three matrix modes are supported:

- **binary**: $M_{i,j} \in \{0, 1\}$ — presence or absence of term i in document j ;
- **tf**: raw term frequency;
- **tfidf**: TF-IDF weight computed with the same formula used by the TF-IDF engine (Section ??).

Boolean queries are evaluated directly on the matrix via a recursive-descent parser embedded in `_evaluate_query()`, which implements the following grammar with correct operator precedence (NOT > AND > OR):

Listing 1: Boolean query grammar (recursive-descent parser).

```

expr      -> and_expr (OR and_expr)*
and_expr  -> not_expr (AND not_expr)*
not_expr  -> NOT not_expr | atom
atom      -> TERM | '(' expr ')'
```

Each grammar rule retrieves the binary term vector for the current token and combines it with the accumulated result using NumPy element-wise operations: AND = multiplication, OR = addition (clipped to 1), NOT = bitwise complement. Space-separated terms without an explicit operator are interpreted as implicit AND.

Results are ranked by the number of distinct query terms matched in each document, providing a natural relevance ordering within the Boolean model.

4.3.4 TF-IDF Engine (`tfidf.py`)

The TF-IDF engine ranks documents by relevance using cosine similarity between TF-IDF vectors. It provides two interchangeable backends selected at construction time:

Custom backend. For each document, token frequencies are computed over the configured fields and weighted by the TF scheme (`raw`, `log`, or `boolean`) and the pre-computed IDF values:

$$\text{idf}_t = \log\left(\frac{N + 1}{\text{df}_t + 1}\right) + 1$$

Document vectors are stored as sparse Python dictionaries. At query time, the query string is preprocessed with the same pipeline as the documents, a sparse query vector is constructed, and cosine similarity is computed against all document vectors in $O(|V|)$ per document.

scikit-learn backend. The same preprocessed token sequences are passed to `TfidfVectorizer` (with `token_pattern=r"\S+"` to respect the custom tokeniser output). The vectorizer fits a TF-IDF matrix over the corpus; at query time the query is transformed into the same feature space and cosine similarity is computed via `sklearn.metrics.pairwise.cosine_similarity`.

Both backends return results as a ranked list of `TFIDFResult` objects sorted by descending score. The engine additionally supports computation of an $N \times N$ pairwise similarity matrix and retrieval of the most similar documents to a given reference document, both via the custom and sklearn backends.

4.3.5 SearchEngine Orchestrator (search_engine.py)

`SearchEngine` is the single entry point for all IR operations. It instantiates and wires together the preprocessing pipeline, the term-document matrix, the Boolean search engine, the inverted index, and the TF-IDF engine, with all components sharing the same preprocessor instance. A single call to `build(documents)` constructs all internal data structures in sequence. The class exposes:

- `boolean_search(query)`: delegates to `BooleanSearchEngine`, which operates on the term-document matrix;
- `tfidf_search(query, top_k)`: delegates to `TFIDFEngine`;
- `author_search(name)`: delegates to the inverted index;
- `intersect(t1, t2)`: exposes skip-pointer-based intersection for the educational panel;
- `similar_to(doc_id, top_k)`: retrieves the most similar documents to a reference;
- `stats()`: returns consolidated statistics from all sub-modules.

All search methods return a normalised `Result` dataclass that provides uniform access to `doc_id`, `document`, `score`, and `source` regardless of the underlying retrieval model.

4.4 REST API (fastapi_app.py)

The REST API is implemented with **FastAPI** and served by Uvicorn. On startup, the API instantiates two `SearchEngine` instances sharing the same document collection but differing in the TF-IDF backend:

- `_engine_custom`: custom TF-IDF implementation;
- `_engine_sklearn`: scikit-learn TF-IDF implementation.

Both engines index the same three fields: `title`, `abstract`, and `full_text`. Document loading is deferred to the first request via `ensure_loaded()`, which calls `load_documents()` to merge the scraper JSON with the PDF full-text JSONs.

Score normalisation is applied to all ranked result lists before returning them to the client: each score is divided by the maximum score in the result set, so the top result always receives a score of 1.0 and the remaining results are proportionally scaled. This makes the displayed percentages more intuitive without altering result ordering.

Table 3 lists the available endpoints.

Tabela 3: REST API endpoints.

Method	Endpoint	Description
GET	/	Redirects to the web interface
GET	/search	Free-text TF-IDF search; supports algorithm, field, year_from, year_to, top_k
GET	/search/boolean	Boolean search (AND / OR / NOT)
GET	/search/author	Author name search (substring, case-insensitive)
GET	/explain	TF-IDF details for a query (educational use)
GET	/index/term/{term}	Postings list and IDF for a vocabulary term
GET	/index/intersect	Skip-pointer intersection of two terms
GET	/matrix/stats	Term-document matrix statistics
GET	/matrix/term/{term}	Binary vector of a term in the matrix
GET	/stats	Consolidated engine statistics

The API is fully documented via FastAPI's automatic OpenAPI/Swagger UI, available at /docs. CORS is unrestricted (`allow_origins=["*"]`) for development convenience. The frontend is served as a static file mount at /ui.

4.5 Web Interface (frontend/index.html)

The frontend is a self-contained single-page application implemented in plain HTML/CSS/JavaScript.



Figura 2: Main dashboard of the developed Web Interface.

It connects to the API at `http://localhost:8000` and provides the following functionality:

- **Search modes:** free-text (TF-IDF), Boolean (AND / OR / NOT), and author search, selectable from the interface;

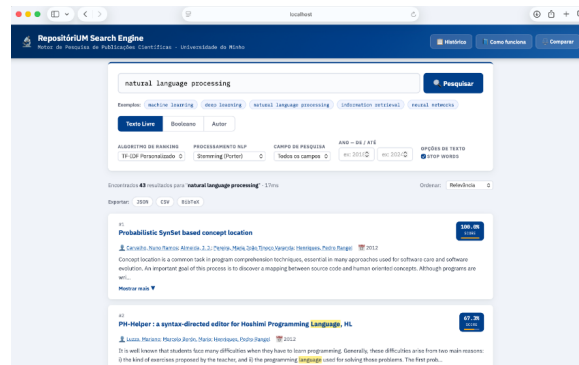


Figura 3: Search results page for the query “deep learning”, showing result cards with title, authors, year, relevance score, and abstract excerpt.

- **Algorithm selection:** the user can choose between the custom and scikit-learn TF-IDF backends; a side-by-side comparison mode runs both simultaneously and displays the results together;
- **Preprocessing options:** stemming vs. lemmatisation, with or without stop-word removal (communicated to the API via query parameters);
- **Field filter:** search can be scoped to all fields, title only, or abstract only;
- **Year filter:** results can be filtered by publication year range;
- **Result cards:** each result displays title, authors, year, abstract snippet, relevance score, DOI, and a link to the full document;
- **Pagination:** results are displayed in pages;
- **Author profile:** searching by author leads to a dedicated author results view;
- **Educational panel:** accessible via the “Como funciona” tab, the panel explains, for example, the TF-IDF and Boolean models through formulas and interactive examples.

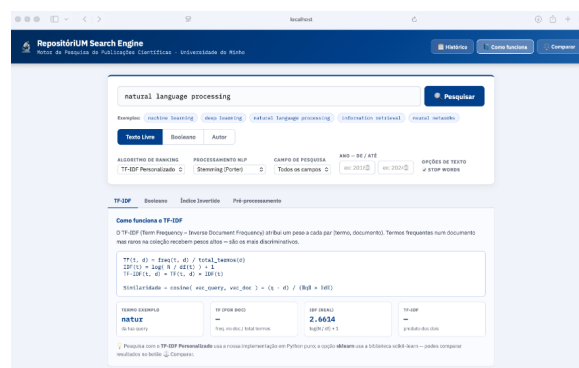


Figura 4: Educational panel explaining the TF-IDF algorithm and displaying real-time term statistics.

4.6 Storage Module (storage/)

The storage module provides an optional SQLite persistence layer managed by the `DocumentStore` class (`database.py`). The relational schema comprises:

- `documents`: core metadata (title, year, DOI, abstract, link);
- `authors` and `document_authors`: normalised author storage with a many-to-many relationship;
- `document_metadata`: flexible key-value store for additional fields;
- `document_versions`: version history;
- `index_terms` and `index_postings`: persistent inverted index;
- `tfidf_weights`: pre-computed TF-IDF weight cache.

This layer provides structured persistence independent of the in-memory indexes, enabling longer-term storage and potential future integration with a database-backed retrieval pipeline.

4.7 Testing

The project includes a comprehensive test suite under the `tests/` directory, covering all major modules:

Tabela 4: Test files and their coverage scope.

Test file	Scope
<code>test_api.py</code>	REST API endpoints (status codes, response format)
<code>test_tfidf.py</code>	TF-IDF computation and cosine similarity
<code>test_index_search.py</code>	Inverted index construction and search
<code>test_query_utils.py</code>	Query parsing and normalisation utilities
<code>test_classifier.py</code>	Naive Bayes topic classifier
<code>test_database.py</code>	SQLite storage layer
<code>test_scraper.py</code>	Scraper metadata extraction

Tests are run with `pytest` and coverage is measured with `pytest-cov`.

5 Limitations

Despite the system fulfilling the core requirements of the project, several limitations should be acknowledged.

- **Collection size:** The indexed collection comprises 78 documents scraped from a single RepositóriUM community. This represents a small fraction of the full repository, which limits the diversity of retrieval results and prevents a statistically meaningful evaluation of ranking quality using standard IR metrics such as Precision@ k , Recall, or nDCG.
- **Full-text coverage:** Full-text PDF content is available for only 20 of the 78 documents. The remaining 58 are indexed solely on title and abstract, which reduces the richness of their TF-IDF vectors and may disadvantage them in ranked retrieval relative to documents with full-text content.
- **Static preprocessing configuration:** Although the frontend exposes options for stemming versus lemmatisation and stop-word removal, these parameters are accepted by the API but do not dynamically reconfigure the retrieval engine at query time. Both engine instances are built once at startup with a fixed stemming configuration. Supporting run-time preprocessing selection would require either multiple pre-built engine instances or on-the-fly index reconstruction.
- **Containerisation:** The system does not yet include a Docker configuration, meaning deployment requires manual environment setup. Adding a `Dockerfile` and `docker-compose.yml` would improve reproducibility and simplify installation across different operating systems.

6 Conclusion

This project resulted in a fully functional search engine for scientific publications applied to RepositóriUM, integrating all the core components of a modern Information Retrieval system.

From a technical standpoint, the most significant challenges were the construction of a robust text preprocessing pipeline capable of handling both English and Portuguese content, the implementation of skip-pointer-based postings list intersection, the integration of two TF-IDF backends with a common interface, and the data fusion between scraper metadata and PDF full-text content. The layered architecture and the use of a unified `SearchEngine` orchestrator kept inter-module dependencies manageable and made it straightforward to expose all functionality through the REST API and the web interface.

The system was validated against the publications from RepositóriUM collection 1822/21295, confirming the correct operation of all retrieval mechanisms. The educational panel in the frontend provides live insight into the preprocessing pipeline and TF-IDF weighting, fulfilling the pedagogical objectives of the project.

As directions for future work, a more rigorous comparative evaluation of the two TF-IDF backends using formal IR metrics such as Precision@ k , Recall, and nDCG would provide a quantitative basis for recommending one implementation over the other. Expanding the document collection to cover a larger portion of RepositóriUM would also allow evaluation of the system's scalability.

References

- [1] Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. Available at: <https://nlp.stanford.edu/IR-book/>
- [2] Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.
- [3] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [4] Ramírez, S. (2019). FastAPI Documentation. Available at: <https://fastapi.tiangolo.com/>
- [5] University of Minho. RepositóriUM — Institutional Repository of the University of Minho. Available at: <https://repositorium.sdum.uminho.pt/>
- [6] DSpace Community (2023). DSpace 8 Documentation. Available at: <https://dspace-labs.github.io/DSpace/>